Geometric Filtering of LLM-Generated Samples for Few-Shot Text Classification

UAI FACULTAD DE INGENIERÍA Y CIENCIAS

benjamin.schindler@alumnos.uai.cl
gonzalo.ruz@uai.cl

Benjamín Schindler **Gonzalo A. Ruz**

Universidad Adolfo Ibáñez, Santiago, Chile
Millennium Nucleus for Social Data Science (SODAS)
Millennium Nucleus in Data Science for Plant Resilience (PhytoLearning)

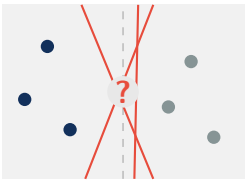
IEEE LACCI 2026

Universidad del Pacífico · Lima, Peru · November 2026

The problem: learning from very few examples

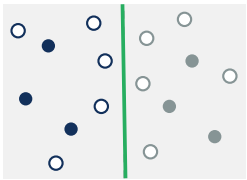
Many real tasks have only **10–50 labeled examples per class**: labeling is slow and costly.

1. Few examples



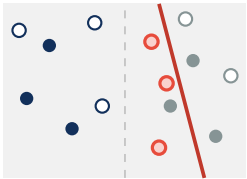
Many boundaries fit the data.
Which one is right?

2. Create more from them



Reliable new examples
pin the boundary down.

3. But only if reliable



Wrong examples
teach the wrong boundary.

real A · real B · new A · new B · new "A" that is really B · – – true boundary

We need more training data, but only data we can trust.

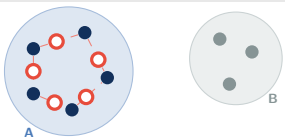
Two ways to create more training data

Classical oversampling (SMOTE)

$x_{\text{new}} = x_i + \lambda\,(x_j - x_i)$ in embedding space

✓ Geometric control

✗ Produces vectors, not text; only fills known space



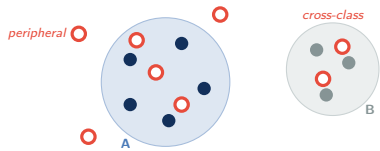
stays on segments between real neighbors

LLM generation (GPT, Gemini, Claude...)

Prompt with real examples, ask for new ones

✓ Fluent, diverse, valid text

✗ No geometric awareness: noise, class overlap



open-ended: no notion of the class region

real, class A real, class B synthetic sample for class A

Can we get LLM-quality text *and* SMOTE-like geometric control?

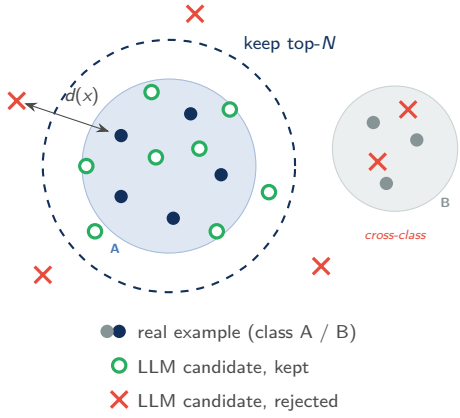
The idea: filter LLM samples in embedding space

LLM samples have **heterogeneous quality**: some land inside their class region, others in peripheral or cross-class zones. Adding all of them can hurt the classifier.

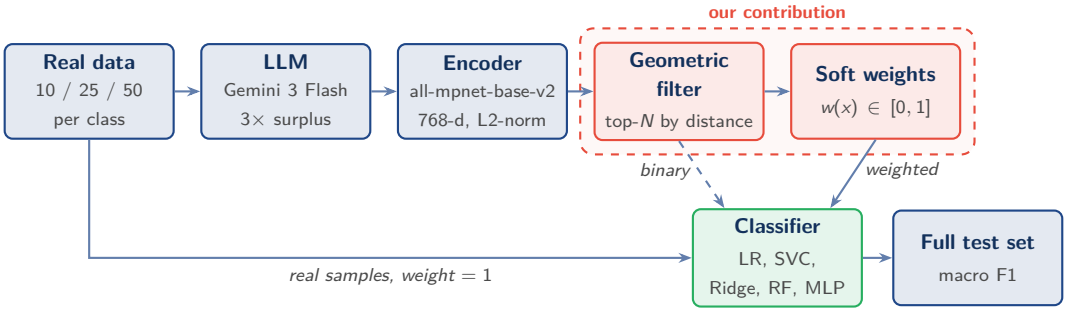
Post-generation geometric filtering:

1. Generate a **3×** **surplus** of candidates with an LLM.
2. Embed them with the same sentence encoder.
3. Score each candidate by its **distance to real examples** of its class.
4. Keep the best; **weight** them by their score.

Research question: can geometric filtering make LLM augmentation reliably better than SMOTE in few-shot settings?



Pipeline: decoupled from the generative model



In-context generation only (no fine-tuning). Any LLM can be plugged in: **the filter only sees embeddings**.

Two contributions: geometric filtering + soft weighting.

Geometric filters, from simple to complex

more criteria

Filter	Criterion
None (control)	Random selection from LLM candidates.
Cascade L1 $\star$	**Euclidean distance to real examples of the class.**
Cascade L2–L4	+ cosine similarity, + kNN purity, + centroid distance (geometric mean).
LOF	Local outlier factor: density vs. real examples.
Combined	Must pass LOF **and** a cosine threshold.

Cascade level 1 score:

$$d(x) = \min_{a \in \mathcal{A}_c} \|x - a\|_2, \qquad s(x) = 1 - \frac{d(x)}{\max d}$$

$\mathcal{A}_c$: real examples of class c.

Candidates are **ranked** and the top-N are kept: no hard threshold to tune.

Spoiler: the simplest filter (L1) wins.

Soft weighting: beyond accept / reject

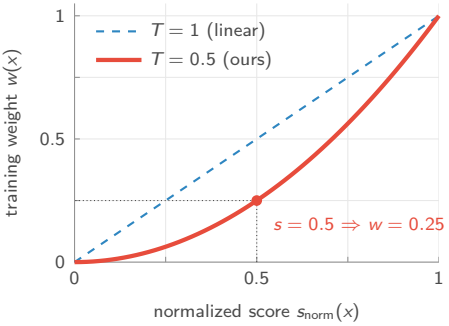
Instead of a binary decision, each kept candidate gets a **training weight**:

1. Compute cascade L1 scores.
2. Keep the top-N candidates.
3. Min-max normalize: $s_{\text{norm}}(x) \in [0, 1]$.
4. Temperature scaling:

$$w(x) = \max\left(w_{\min},\ s_{\text{norm}}(x)^{1/T}\right)$$

$T = 0.5$ (exponent 2), $w_{\min} = 0$. Real samples always get $w = 1$.

Weights are passed to the classifier as `sample_weight`.



$T < 1$ concentrates weight on the most class-consistent candidates.

Geometrically consistent samples contribute more to training.

Experimental setup: 6,700+ configurations

13	10	5	5	3
datasets	methods	classifiers	LLMs	*n*-shot levels

Data [9 domains; 10 / 25 / 50 per class; full test sets]

- **7 classification** (2–20 classes): sms_spam, hate_speech, 20newsgroups, ag_news, emotion, dbpedia14, 20newsgroups (20 cl.)
- **3 scalability** (6–150 classes): trec6, banking77, clinc150
- **3 NER**: MultiNERD, WikiANN, Few-NERD

Methods [5 classifiers, 5 LLMs from 4 providers]

- Ours (binary, soft) vs. SMOTE, random oversampling, EDA, back-translation, T5 paraphrase, BERT contextual, embedding mixup, no augmentation.

Statistics

- Metric: **macro F1**; Δ vs. SMOTE in pp.
- Paired *t*-tests, **Bonferroni**-corrected.
- Cohen's *d*, bootstrap 95% CI (10k resamples).
- **Win rate**: % of configurations beating SMOTE.

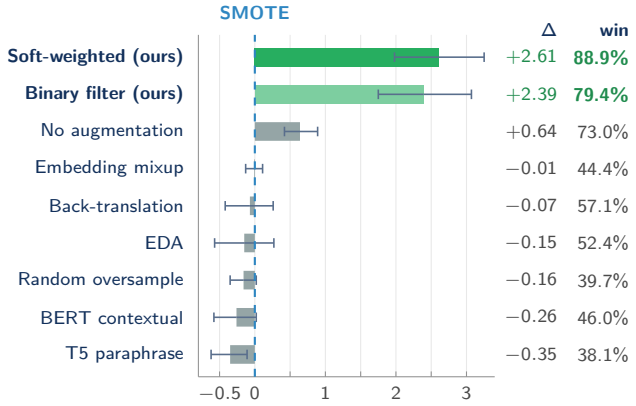
Main comparison

21 dataset configs $\times$ 3 linear classifiers $\times$ 3 seeds $\times$ 10 methods = **1,890 runs**

Main result: +2.61 pp over SMOTE

+2.61 pp
macro F1 vs. SMOTE
(soft-weighted)

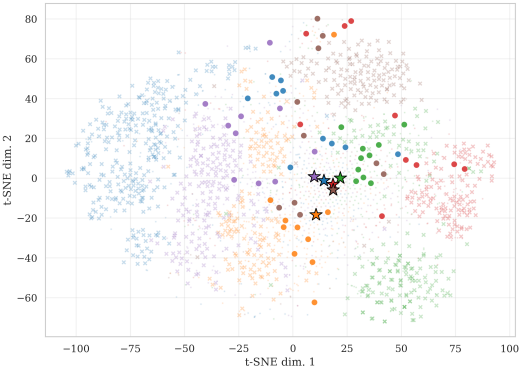
- $p < 0.0001$ [Bonferroni]
- Cohen's $d = 0.95$ [large]
- Win rate $= 88.9\%$
- 95% CI $[+1.98,\ +3.25]$



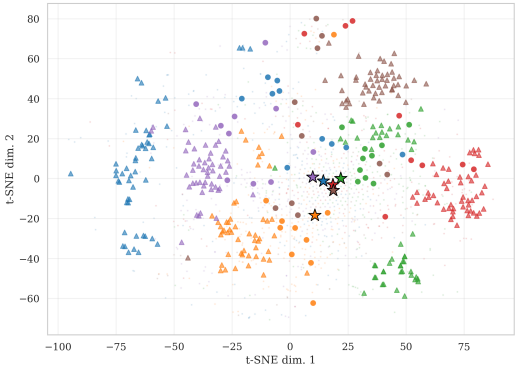
Modern baselines (T5, BERT, mixup) do *not* beat SMOTE; filtered LLM data does.

What the filter does: emotion dataset, 10-shot

Before: all LLM candidates (×)



After: kept by cascade L1 (▲)



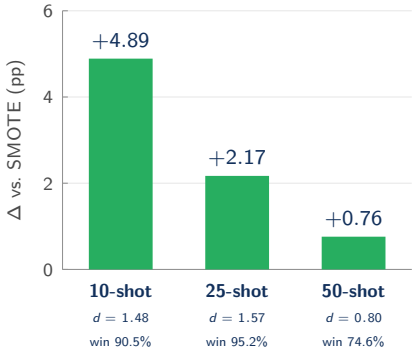
anger fear joy love sadness surprise

● real × candidate ▲ kept ★ centroid

Cross-class and peripheral candidates are discarded; class regions get denser.

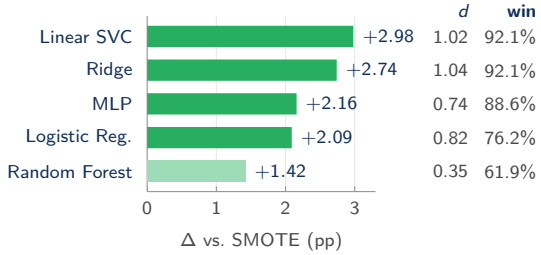
Largest gains where data is scarcest

By examples per class



10-shot macro F1: **67.23%** → **72.12%**

By classifier



Linear models gain most: filtered samples densify class regions near hyperplane boundaries. Axis-parallel tree splits benefit less.

All gains significant; strongest at 10-shot and for linear classifiers.

Counter-intuitive finding: simple beats complex

1. Redundant metrics

For L2-normalized vectors:

$$\|u - v\|_2^2 = 2\,(1 - \cos(u, v))$$

Cosine adds **no new information**: same ranking.

2. LOF breaks at few-shot

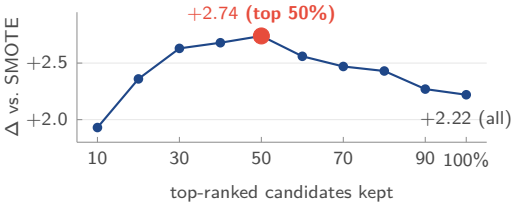
LOF uses $k = 20$ neighbors, but at 10-shot a class has only **9 real neighbors**.

Density estimates degenerate; distance stays stable.

3. Over-filtering

Intersecting criteria keeps only **near-duplicates**.

Combined filter: 0% acceptance in 6 of 9 NER configs; the only filter not significant.



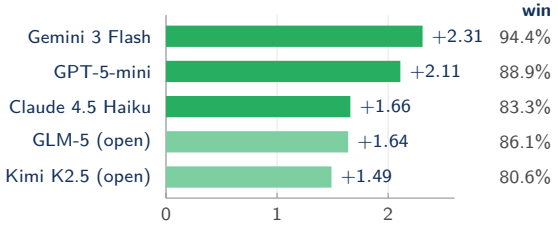
Curriculum check: adding candidates best-first, performance **peaks at the top 50%** and then declines.

The lower half of the ranking adds noise, not signal.

Generate a surplus, filter lightly by distance. More filtering is not better.

Robust across LLMs, and it transfers to NER

By generator LLM [all significant]



Friedman $\chi^2 = 8.15$, $p = 0.086$: no significant LLM effect.

Kendall $W = 0.59$: models agree on which configs benefit.

NER, same filter, **no modification**

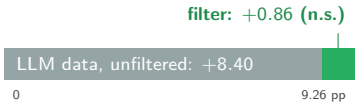
MultiNERD, WikiANN, Few-NERD; class = dominant entity.

+9.26 pp

entity-level F1 vs. no augmentation

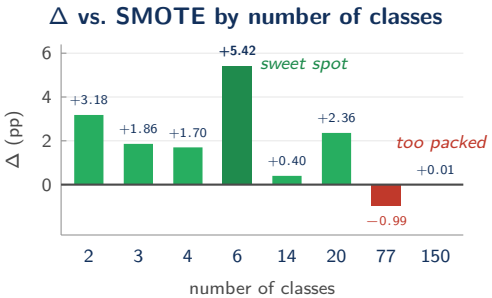
$p_{\text{Bonf}} = 0.002$, $d = 2.05$, **100% win rate** (9/9)

Where the +9.26 pp come from



The filter is decoupled from the LLM and from the task.

When does it help? And where are the limits?



Spearman $\rho = -0.26$, $p = 0.003$

Why it works: cleaner geometry

Tighter classes (intra-class distance 0.822 vs. 0.850) and better separation (inter-class 0.616 vs. 0.571), both $p < 0.001$.

Helps most

- **Hard classes** (F1 $< 30\%$): $+10.44$ pp vs. $+0.55$ pp for easy ones (F1 $> 80\%$). *anger*, emotion 10-shot: $27.8\% \rightarrow 44.6\%$.
- **Few classes**: best at 6 ($+5.42$ pp, 100% win rate).
- **10–25 per class**, linear classifier.

Limitations

- **Many classes**: -0.99 pp at 77, $+0.01$ pp at 150.
- **More data**: only $+0.76$ pp at 50-shot.
- **English only**; multilingual transfer untested.
- **Same encoder** filters and classifies; mitigated by 90.5% agreement across 4 embedding models.

Take-home messages

1. **Distance filtering works:** +2.61 pp over SMOTE ($d = 0.95$, 88.9% win rate) across 13 datasets.
2. **Simple beats complex:** distance alone outperforms LOF, cosine and combined criteria. Don't over-filter.
3. **Decoupled and general:** robust across 5 LLMs; transfers to NER (+9.26 pp) unchanged.

Next: multilingual embeddings, local open-source LLMs, joint tuning of filter and weighting, other tasks (QA, summarization).

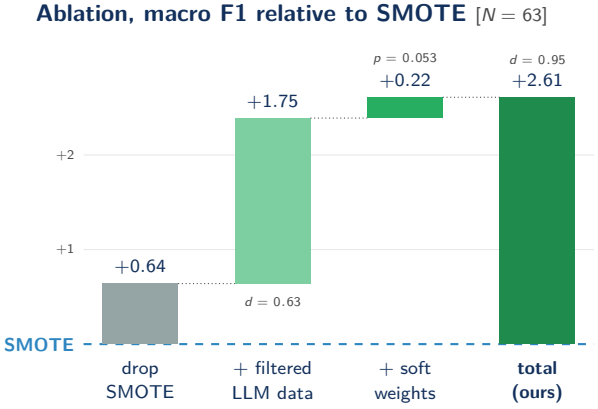
Recipe

1. Prompt an LLM with your few real examples.
2. Generate a **3× surplus**.
3. Embed; score by **distance to the nearest real example**.
4. Keep the top half; **soft-weight** them.
5. Train a linear classifier.

Thank you! Questions?

benjamin.schindler@alumnos.uai.cl · gonzalo.ruz@uai.cl

Backup: where does the gain come from?



Most of the contribution comes from **geometric selection itself** (+1.75 pp over no augmentation, $p < 0.0001$); soft weighting adds a small, marginally significant refinement (+0.22 pp).

Binary filter vs. SMOTE: +2.39 pp, $d = 0.82$.
No augmentation already beats SMOTE by +0.64 pp.

Caching caveat: cached LLM responses give zero cross-seed variance for deterministic classifiers (80% of configs). Restricted to stochastic classifiers the effect remains: +1.79 pp, $p = 0.0015$, $d = 0.52$.

Backup: NER results (entity-level F1)

Corpus	n	Baseline	No filter	Cascade L1	LOF
MultiNERD	10	.247	.393	.407	**.426**
	25	.315	.419	**.438**	.431
	50	.415	.486	.455	**.495**
WikiANN	10	.162	.268	**.303**	.302
	25	.253	.272	**.299**	.276
	50	.244	**.295**	.287	.293
Few-NERD	10	.103	**.234**	.220	.217
	25	.161	.240	**.251**	.235
	50	.200	.248	**.272**	.261
Average		.233	.317	.326	.326
Δ vs. baseline		—	+8.40	+9.26	+9.28

Cascade L1: $p_{\text{Bonf}} = 0.002$ (7 filters), $d = 2.05$, 100% win rate. Combined filter: 0% acceptance in 6/9 configs, $p = 0.29$.

Backup: protocol details

Generation

- Gemini 3 Flash (primary); GPT-5-mini, Claude 4.5 Haiku, Kimi K2.5, GLM-5.
- Real examples as in-context demonstrations; 3 candidates per desired sample.
- Responses cached (MD5) for reproducibility.

Embeddings

- all-mpnet-base-v2, 768-d, L2-normalized.
- Ablation with bge-large, e5-large, bge-small: 90.5% cross-model agreement.

Protocols

- Main: 10 methods, 3 linear classifiers, 3 seeds (1,890 runs).
- Extended: 7 methods, 5 classifiers, 5 seeds (3,675 runs); MLP and RF rows.
- Seeds: 42, 123, 456, 789, 1011.

Classifiers

- Logistic regression, linear SVC, Ridge, random forest (100 trees), MLP (100 hidden units).

Teacher-model filtering

- Geometric $+2.78$ vs. teacher $+2.43$ pp (binary variants); paired $p = 0.0001$.